CO326 - Embedded Systems & Applications

Cooling Tower Fan Degradation Monitoring & Control System

Project Report

GROUP - 08

M.L De Croos Rubin - E/20/054

A.H.D. Kawya - E/20/197

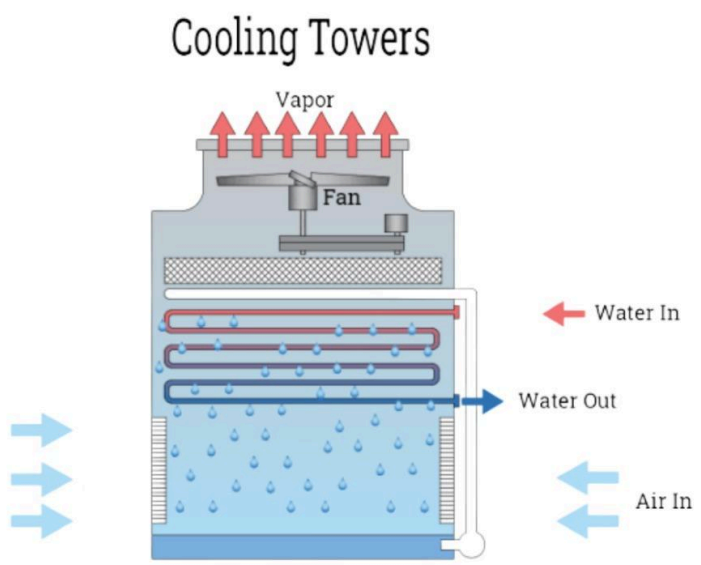
K.W.I.T. Kumarasinghe - E/20/211

D.M.B. Edirisooriya - E/20/093

1. Introduction (E/20/093)

Cooling towers are critical components in industrial plants, HVAC systems, and power generation facilities. The mechanical health of the cooling tower fan directly determines the efficiency and reliability of the entire cooling circuit. Blocked blades, rotor imbalance, and bearing degradation can cause overheating, downstream equipment damage, and costly unplanned downtime if left undetected.

This project addresses this problem by implementing a complete Edge-to-Cloud Cyber-Physical System (CPS) and Digital Twin for cooling tower fan health monitoring. The system captures real-time vibration data at the edge, performs on-device AI classification, calculates Remaining Useful Life (RUL), and exposes full bidirectional control through a SCADA-style dashboard , shifting the maintenance strategy from reactive (fix after failure) to predictive (intervene before failure).

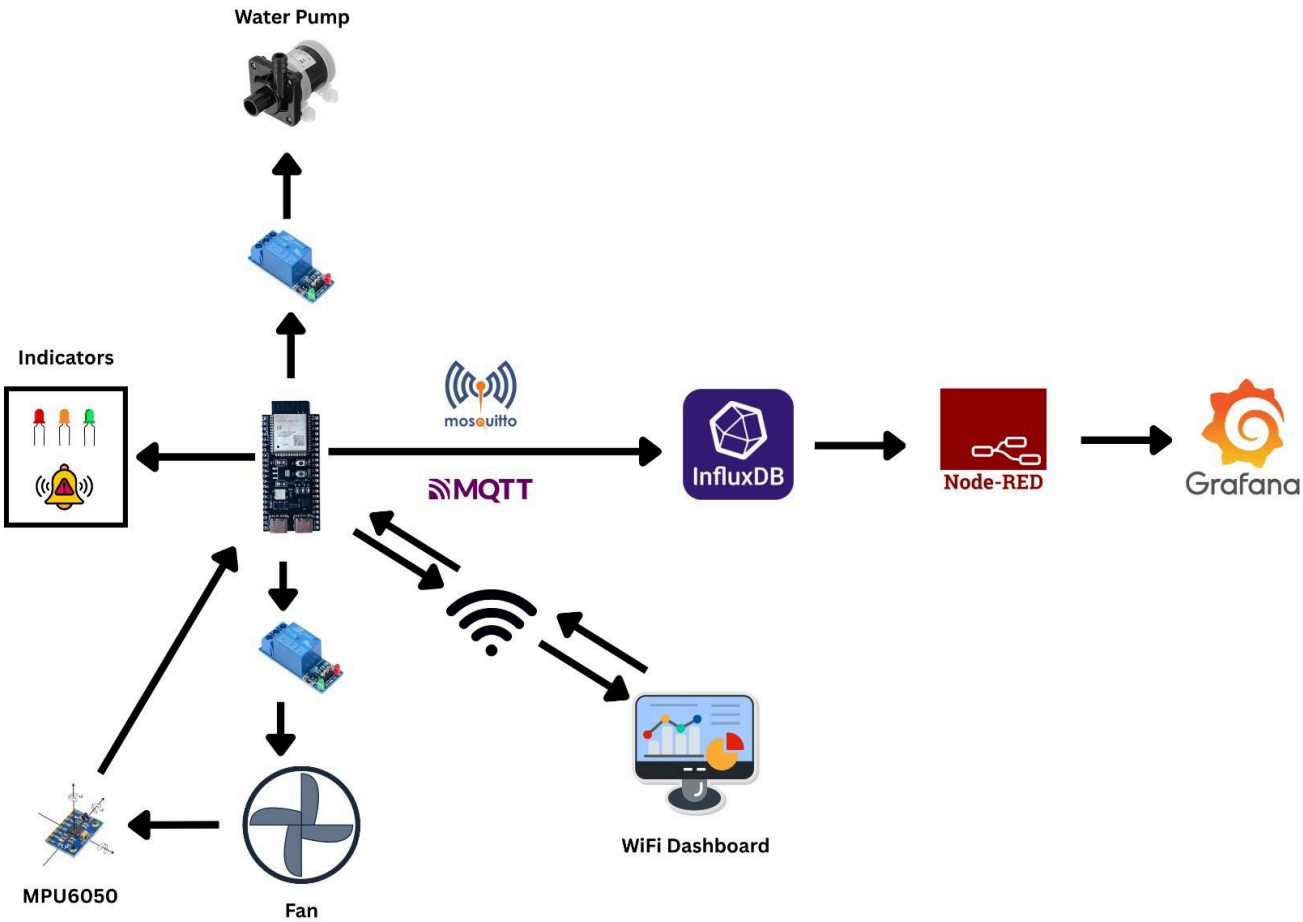


2. Project Objectives (E/20/093)

- Classify fan operating state in real time (Normal, Blocked, Unbalanced, Off) using vibration sensing and a classification model running entirely on the ESP32-S3.
- Estimate Remaining Useful Life (RUL) by tracking the trend of mean vibration energy over time using linear regression in Node-RED.
- Implement a hardware safety interlock that automatically shuts down the pump relay on any fault state and auto-restarts it when the fan recovers to Normal.
- Provide bidirectional MQTT control so operators can command both the fan and pump relays remotely from Node-RED, Grafana, or any MQTT client.
- Persist all time-series telemetry in InfluxDB for historical degradation analysis over days or weeks.
- Visualize system health in a Grafana SCADA-style Digital Twin dashboard with live alerts.

3. System Architecture (E/20/054)

The system follows the 4-Layer Industrial IoT architecture, where each layer has a distinct responsibility and communicates over open, standardized protocols.



Layer	Technology	Key Responsibilities
Perception & Edge	ESP32-S3 DevKitC-1 + MPU6050	Vibration sampling at 500 Hz, FFT feature extraction, classification model inference, relay switching, LED & buzzer indicators, local HTTP dashboard
Transport	Eclipse Mosquitto (MQTT Broker)	Lightweight pub-sub messaging using Sparkplug B Unified Namespace topic structure. Last Will & Testament (LWT) for connection reliability
Edge Logic	Node-RED	JSON parsing, RUL linear degradation thresholds, alert routing, emergency stop handling, MQTT command forwarding
Application	InfluxDB v2.7 + Grafana	Time-series historian for historical trend analysis; SCADA-style Digital Twin dashboard with live gauges, trend charts, fault log.

4. Hardware Design (E/20/054)

4.1 Components

Component	Qty	Role
ESP32-S3 DevKitC-1 (N16R8) - 16 MB Flash, 8 MB PSRAM	1	Main microcontroller, Edge AI, Wi-Fi
MPU6050 6-axis IMU (Accelerometer + Gyroscope)	1	Vibration sensing via I2C
5V Relay Module JQC3F (transistor-driven, active-HIGH)	2	Fan and pump circuit switching
12V DC Cooling Fan + 12V Adapter	1	Cooling tower fan under test
12V DC Water Pump + 12V Adapter	1	Cooling circuit pump
Green LED + 220 Ω resistor	1	Normal state indicator
Red LED + 220 Ω resistor (×2)	2	Blocked / Unbalanced fault indicators
Active Buzzer	1	Audible fault alarm
Breadboard + Jumper Wires	1 set	Prototyping connections

4.2 GPIO Pin Assignment

GPIO	Connected To	Direction	Notes
GPIO 8	MPU6050 SDA	Bidirectional	I2C data - 3.3 V logic
GPIO 9	MPU6050 SCL	Output	I2C clock
GPIO 4	Green LED (+)	Output	Normal state
GPIO 5	Red LED 1 (+)	Output	Blocked fault
GPIO 6	Red LED 2 (+)	Output	Unbalanced fault
GPIO 7	Relay 1 IN (Fan)	Output	Active-HIGH; HIGH = relay ON
GPIO 15	Relay 2 IN (Pump)	Output	Active-HIGH; HIGH = relay ON
GPIO 16	Buzzer (+)	Output	Double beep on fault

4.3 Relay Wiring

The JQC3F relay module is transistor-driven and active-HIGH, unlike the more common active-LOW opto-isolated modules. VCC is connected to 5 V ,GND is connected to a common ground. The firmware macros FAN_ON() = HIGH and FAN_OFF() = LOW reflect this. For each relay, COM connects to the positive terminal of the respective 12 V adapter, and NO (Normally Open) connects to the positive wire of the fan or pump. All negative wires return directly to the adapter ground.

5. Firmware Design (ESP32-S3) (E/20/197)

5.1 Vibration Signal Processing Pipeline

Every inference cycle follows a fixed, deterministic pipeline:

1. 1128 acceleration samples are collected at exactly 500 Hz (2000 µs interval via delayMicroseconds).
2. Resultant vector magnitude is computed per sample: $|a| = \sqrt{(ax^2 + ay^2 + az^2)}$, making the measurement orientation-independent.
3. A Hamming window is applied before the FFT to reduce spectral leakage at bin edges.
4. 128-point FFT (ArduinoFFT library) transforms the time-domain signal to the frequency domain.
5. Three features are extracted from the first 64 bins (DC excluded): Dominant Frequency (Hz), Peak Amplitude, and Mean Spectral Energy.

5.2 Edge AI – Classification model

Dataset & Feature Extraction

The training pipeline loads vibration data from fan_dataset.csv, where each row contains 128 raw accelerometer magnitude samples and a class label. Before training, every window is passed through the same FFT feature extraction function used in firmware - 128-point FFT at 500 Hz, DC bin skipped, yielding three features: dominant frequency (Hz), peak amplitude, and mean spectral energy. This guarantees zero training-serving skew. The dataset is split 80/20 into training and test sets using stratified sampling to preserve class balance across all four classes.

Model 1 - Decision Tree

The Decision Tree classifier (max_depth=6) is trained using scikit-learn's DecisionTreeClassifier. After training, the tree is exported to dt_model.h as a nested if-else C++ function (dt_predict) using micromlgen, or a manual recursive generator if micromlgen is unavailable. The exported function takes the three raw features directly, no scaler is needed and returns a class integer. It runs entirely on the ESP32-S3 with minimal memory and no floating-point overhead beyond the feature extraction itself.

Model 2 - Random Forest

The Random Forest (15 estimators, max_depth=6) is trained using RandomForestClassifier. It aggregates votes from 15 independent decision trees, offering higher robustness against noisy vibration readings compared to a single tree. The ensemble is exported to rf_model.h via micromlgen as a collection of C++ if-else trees. The tradeoff is a larger code footprint than the single Decision Tree, but still well within the ESP32-S3's 16 MB flash.

Model 3 - Neural Network

The neural network (architecture: 3 → 16 → 8 → 4) is built with Keras and trained for 150 epochs using the Adam optimiser with categorical cross-entropy loss. A StandardScaler is fit on the training features and its mean and standard deviation are exported alongside the weights. At inference time on the ESP32, the three raw features are first normalised using the scaler parameters, then passed through two ReLU hidden layers and a Softmax output layer in a hand-rolled forward pass. All weights, biases, and scaler parameters are stored in nn_model.h as const float arrays. The classifier runs entirely on the ESP32-S3, no network connection is required for classification.

Class	Label	Meaning	Safe for Pump?
0	NORMAL	Fan running correctly, no faults detected	Yes
1	BLOCKED	Fan blades obstructed - high vibration energy	No
2	UNBALANCED	Rotor imbalance - elevated vibration at rotation frequency	No
4	OFF	Fan not running	No

5.3 Safety Interlock Logic

Only Class 0 (NORMAL) is considered safe. Any other classification triggers the pump interlock.

- **Fault Detected:** Fan NOT Normal → pump relay immediately de-energises. The pumpWasRunning flag records whether the pump was active before the fault.
- **Recovery:** Fan returns to Normal → interlock releases; pump auto-restarts if pumpWasRunning is true.
- **Start Guard:** Manual pump start command → denied unless currentPrediction == 0. A clear warning is printed to Serial.

5.4 Indicators and Alarms

Indicator	Condition	Behaviour
Green LED (GPIO 4)	Class 0 - Normal	ON
Red LED 1 (GPIO 5)	Class 1 - Blocked	ON
Red LED 2 (GPIO 6)	Class 2 - Unbalanced	ON
All LEDs	Class 4 - Off	All OFF
Active Buzzer (GPIO 16)	Class 1 or 2 (any fault)	Double-beep

5.5 Bidirectional MQTT Communication

The ESP32 maintains a non-blocking MQTT connection using PubSubClient. It publishes a JSON telemetry payload after every 2-second inference cycle and subscribes to two command topics. The MQTT reconnect function is non-blocking, it attempts one reconnect per loop iteration rather than looping until connected, ensuring the local web dashboard stays responsive even when the broker is offline.

Topic	Direction	Payload
sensors/group08/coolingTower/data/fan01	ESP32 → Broker	JSON: fan_state, dom_freq, max_amplitude, mean_energy, fan_running, pump_running, pump_interlocked, class
sensors/group08/coolingTower/cmd/fan01	Broker → ESP32	ON | 1 - fan on; OFF | 0 | SHUTDOWN - fan off
sensors/group08/coolingTower/cmd/motor01	Broker → ESP32	ON | 3 | 4 - pump on; OFF | 2 | SHUTDOWN - pump off

5.6 Local Web Dashboard

An HTTP web server (port 80) serves a responsive dashboard at the ESP32's local IP address. The page auto-refreshes every 3 seconds and shows the current fan classification, fan and pump relay states, interlock status, and toggle buttons for all relays. This dashboard is fully self-contained and operational even if the MQTT broker or cloud stack is unreachable.

6. Cloud Infrastructure (E/20/211)

All server-side components run in Docker containers orchestrated by a single docker-compose.yml file. Start the entire stack with docker-compose up -d. Each service stores its data in a named volume for persistence across restarts.

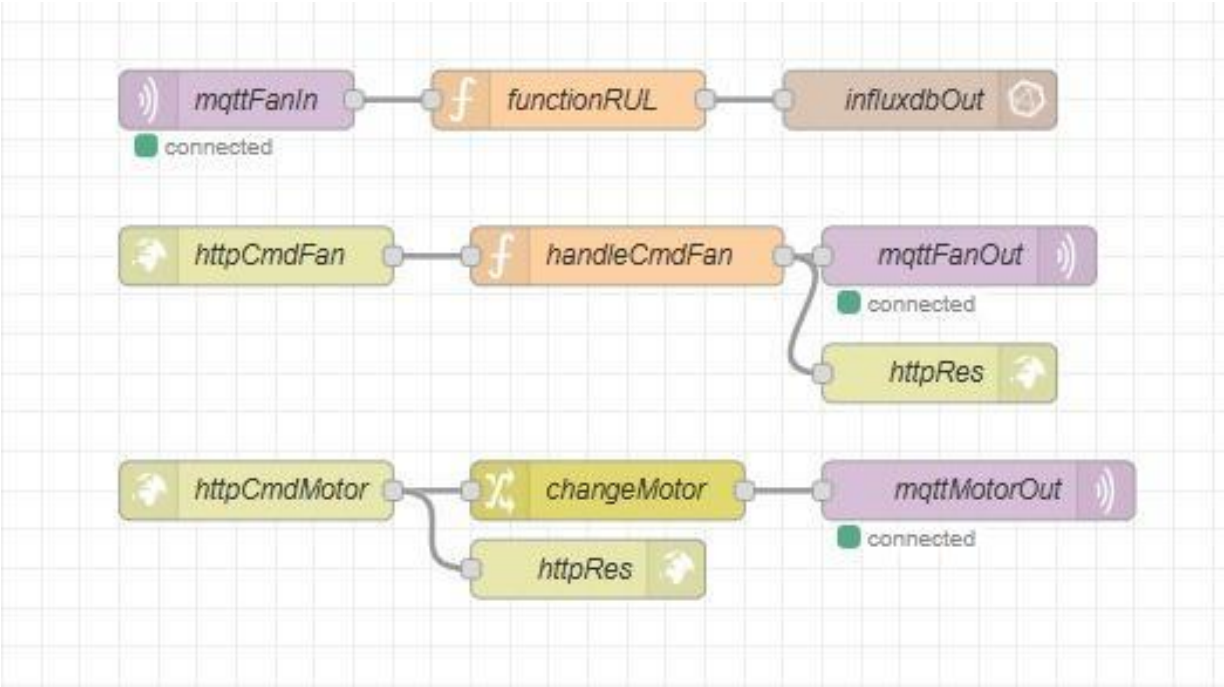
Service	Image	Port	Volume
Mosquitto	eclipse-mosquitto:latest	1883	./mosquitto/{config, data, log}

Node-RED	nodered/node-red:latest	1880	./nodered/data
InfluxDB	influxdb:2.7	8086	./influxdb/data
Grafana	grafana/grafana:latest	3000	./grafana/data

6.1 Mosquitto MQTT Broker

Mosquitto is configured with “allow_anonymous: true” to permit connections from all devices on the local network. The Sparkplug B Unified Namespace (UNS) topic structure is used for topic naming. Last Will & Testament (LWT) is configured so that if the ESP32 disconnects unexpectedly, a retained OFFLINE message is published to notify subscribers.

6.2 Node-RED - Logic & RUL Calculation



Node-RED acts as the integration hub between the edge device, the historian, and the dashboard. The main flow comprises:

- MQTT In node subscribes to the telemetry topic; JSON node parses the raw string into a JavaScript object.
- RUL Function node maintains a rolling buffer of mean_energy readings. Linear regression is computed on the buffer: y = mx + c. If slope m > 0 (energy increasing), RUL (hours) = (failure_threshold - current_energy) / (m × readings_per_hour). A flat or negative slope reports a high healthy RUL value.
- InfluxDB Out node writes all telemetry fields plus the computed RUL to the coolingTower measurement.

- Alert node monitors fan_state; BLOCKED or UNBALANCED states trigger warning messages forwarded to Grafana via a retained MQTT topic.
- Emergency Stop: an HTTP In node receives POST requests from Grafana's emergency stop button. A Change node forces the payload to SHUTDOWN, and two MQTT Out nodes publish to both command topics, de-energising both relays within MQTT round-trip time (< 100 ms on a local network).

6.3 InfluxDB v2.7 - Time-Series Historian

InfluxDB stores every telemetry reading as a timestamped data point in the coolingTower measurement. Unlike a relational database, InfluxDB is optimised for high-write-rate time-stamped data. Storing readings persistently is what enables Grafana to plot degradation trends over days or weeks, not just the current snapshot.

Field	Type	Description
fan_state	String	Classification label (NORMAL, BLOCKED, UNBALANCED, EMPTY, OFF)
class	Integer	Classification index (0-4)
dom_freq	Float	Dominant FFT frequency (Hz)
max_amplitude	Float	Peak spectral amplitude
mean_energy	Float	Mean spectral energy - primary degradation indicator
fan_running	Boolean	Fan relay state
pump_running	Boolean	Pump relay state
pump_interlocked	Boolean	Safety interlock active flag
rul_hours	Float	Estimated Remaining Useful Life (hours)

6.4 Grafana - SCADA-Style Digital Twin Dashboard

The Grafana dashboard is the operator-facing interface. It is designed to resemble an industrial SCADA screen, giving immediate situational awareness and historical context:

- Current Fan State panel: colour-coded Stat panel (green = Normal, red = any fault) showing the latest classification label.
- Vibration Energy Trend: Time Series chart of mean_energy over a configurable window (1 h / 24 h / 7 days) the primary degradation timeline.
- Dominant Frequency Trend: Time Series chart of dom_freq, revealing rotational frequency shifts characteristic of mechanical faults.
- RUL Gauge: colour-threshold gauge (green → yellow → red) showing estimated operational hours remaining.
- Fault Event Log: table panel querying all records where class ≠ 0, providing a timestamped history of every fault event.
- Fan & Pump Status: Stat panels for relay state and interlock flag.

7. System Control Flow (E/20/197)

The following table summarises all control paths, from source to physical actuator across every interface the system supports.

Source	Command	Path	Outcome
Serial Monitor	'1' / '0'	UART → firmware setFan()	Fan relay ON / OFF
Serial Monitor	'3' / '2'	UART → firmware setPump()	Pump ON (if Normal) / OFF
Local Web Dashboard	Button click	HTTP GET /fan/on etc. → firmware	Relay toggle (pump denied if not Normal)
MQTT Client / Node-RED	ON / SHUTDOWN	MQTT cmd topic → mqttCallback()	Relay toggle
Grafana Emergency Stop	Button click	HTTP POST → Node-RED → MQTT → ESP32	Both relays OFF within < 100 ms
Interlock (automatic)	Fault detected	Firmware updateInterlock()	Pump OFF; auto-restart on recovery

8. Setup & Deployment (E/20/093)

8.1 Firmware Upload

6. Install Arduino IDE. Set board to ESP32S3 Dev Module with USB CDC On Boot: Enabled, Flash Size: 16 MB, PSRAM: OPI PSRAM.
7. Install libraries: MPU6050 (Electronic Cats) and arduinoFFT v2.x.
8. Place nn_model.h in the same folder as fanalyzer_neural_network.ino.

9. Edit WIFI_SSID, WIFI_PASSWORD, and mqtt_server (host PC IP) at the top of the .ino file.
10. Upload via USB. Open Serial Monitor at 115200 baud to verify connection and obtain the ESP32's IP address.

8.2 Cloud Stack

11. Ensure Docker and Docker Compose are installed on the host PC.
12. Clone the repository: https://github.com/cepdnaclk/e20-co326-group08-Industrial-Fan-Vibration-Based-Predictive-Maintenance
13. From the project root, run: docker-compose up -d
14. Access: Node-RED at localhost:1880 | InfluxDB at localhost:8086 | Grafana at localhost:3000

9. Conclusion (E/20/197)

This project successfully delivers a complete, production-representative predictive maintenance system for an industrial cooling tower fan. Edge AI classification on the ESP32-S3 identifies fan fault states in real time with no cloud dependency, while a hardware safety interlock automatically protects the pump from operating into a degraded cooling circuit. The full IoT stack - MQTT, Node-RED, InfluxDB, and Grafana, provides persistent historical storage, automated RUL estimation, and a SCADA-style digital twin dashboard with bidirectional remote control and a one-click emergency stop.

The system demonstrates how a low-cost embedded platform combined with open-source cloud tooling can implement industrial-grade predictive maintenance at a fraction of the cost of traditional proprietary SCADA solutions, and serves as a solid foundation for future extensions such as multi-fan monitoring, TLS-secured MQTT, OTA firmware updates, and more sophisticated degradation models.

10. References

1. Delta Cooling Towers Inc. (2017). *What is a Cooling Tower? | Manufacturers of Cooling Towers & Systems | Delta Cooling Towers, Inc.* [online] Manufacturers of Cooling Towers & Systems by Delta Cooling Towers, Inc. Available at: https://deltacooling.com/resources/faqs/what-is-a-cooling-tower.
2. Espressif.com. (2025). *Espressif Documentation*. [online] Available at: https://documentation.espressif.com/esp32-s3_datasheet_en.pdf.
3. alldatasheet.com (2021). *MPU-6050 PDF*. [online] Alldatasheet.com. Available at: https://www.alldatasheet.com/datasheet-pdf/view/1132807/TDK/MPU-6050.html.
4. Youtu.be. (2026). Available at: https://youtu.be/dwi40PFO8HQ?si=tDyY2BtTNGEpX8pS.
5. Youtu.be. (2026). Available at: https://youtu.be/7t62iu3W7Y4?si=iRQHN1AXyOZ_uN0b.
6. https://www.facebook.com/Industrial-Cooling-Solutions-Inc-ICS-807464446083414 (2025). *5 Signs of Cooling Tower Failure You Need to Address - New Cooling Tower Construction, Parts, Maintenance, Upgrades*. [online] New Cooling Tower Construction, Parts, Maintenance, Upgrades. Available at: https://h2ocooling.com/signs-of-cooling-tower-failure/.

11. Individual Contribution

M.L De Croos Rubin - E/20/054
Initial hardware setup, firmware development, Testing and validation, data collection, training Edge AI models.

A.H.D. Kawya - E/20/197
Initial hardware setup, firmware development, Testing and validation, data collection, training Edge AI models.

K.W.I.T. Kumarasinghe - E/20/211
MQTT broker configuration, Node-RED flow development, Grafana dashboard, InfluxD setup.

D.M.B. Edirisooriya - E/20/093
Testing and validation, firmware development, data collection, training Edge AI models.